

Companion to Bird & de Moor, Algebra of Programming

§9. Dynamic Programming

§9.1. Theory

§9.1.1. Example: segmenting a list

- $(\mathsf{T}) = \text{concat}$
 $\mathsf{T} = [\text{nil}, \text{cat}]$ takes a non-empty first list, so T° cuts a non-empty prefix off a list every way, and folding with T flattens a list of segments
- $(\mathsf{h}) = \text{list}(\text{sum})$
 $\mathsf{h} = [\text{nil}, \text{cons}(\text{sum} \times 1)]$ puts a segment's sum in front, so folding with h sums every segment
- $\mathsf{H} = \text{partition list}(\text{sum})$
 H segments a list every way, then sums each segment; with R comparing the largest entry, M cuts the list so that the largest segment sum is as small as possible

(9.1.1a)

(9.1.1b)

$\mathbf{x}$	$\mathbf{h(x)}$
$\text{inl}(())$	$[\]$
$\text{inr}([c], [])$	$[c]$
$\text{inr}([a, b], [c])$	$[a + b, c]$
$\text{inr}([a], [b + c])$	$[a, b + c]$

- $\mathsf{H}([a, b, c], \mathsf{ys}) \iff \mathsf{ys} = [a + b + c] \vee \mathsf{ys} = [a, b + c] \vee \mathsf{ys} = [a + b, c] \vee \mathsf{ys} = [a, b, c]$
 H sends $[a, b, c]$ to the sums of its four segmentations and nothing else
- $\mathsf{T}^\circ \mathsf{F}(\mathsf{H}) \mathsf{h} = \mathsf{H}$
cut once, solve the rest, put the pieces back together

(9.1.1c)

(9.1.1d)

$\mathsf{h} : \mathsf{FB} \rightarrow \mathsf{B}$ a map, $\mathsf{T} : \mathsf{FA} \rightarrow \mathsf{A}$ an F -algebra, $\mathsf{R} : \mathsf{B} \rightarrow \mathsf{B}$.

$\mathsf{H} \triangleq (\mathsf{T})^\circ (\mathsf{h}) : \mathsf{A} \rightarrow \mathsf{B}$, $\mathsf{M} \triangleq \frac{\mathsf{H}}{\mathsf{S}} \text{est}(\mathsf{R})$ the problem to be solved, $(\mu \mathsf{X} : \mathsf{G}(\mathsf{X}))$ as in $\triangleq \mu$.

$\mathsf{DPSetting}(\mathsf{h}, \mathsf{R}) \triangleq \text{Map}(\mathsf{h}) \wedge \text{MonoAlg}(\mathsf{h}, \mathsf{R}^\circ) \wedge \mathsf{R}^\circ \mathsf{R}^\circ \sqsubseteq \mathsf{R}^\circ$

h is a map, h is monotonic on R° , and R° is transitive — assumed by every theorem below

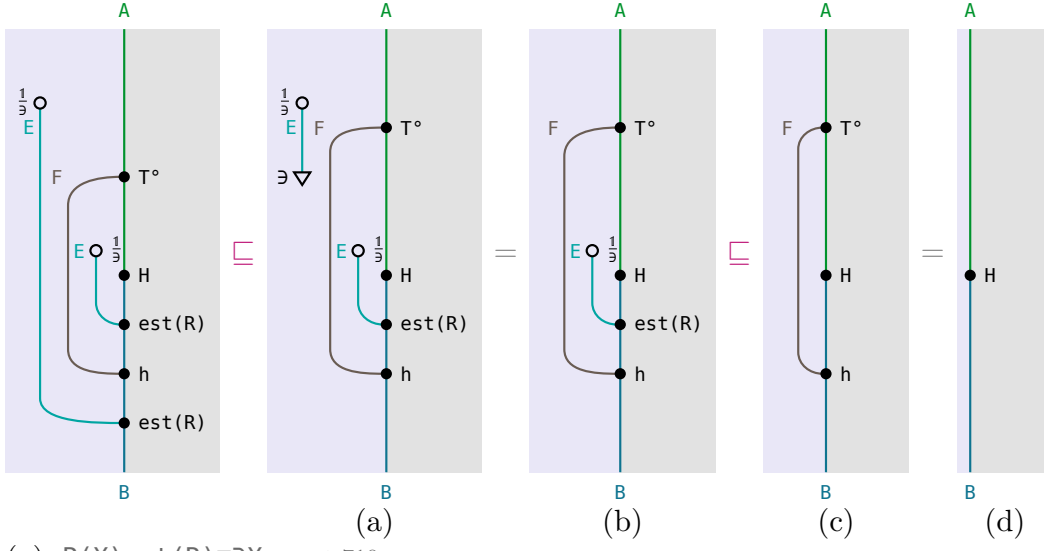
$(\mu \mathsf{X} : \frac{\mathsf{T}^\circ}{\mathsf{S}} \mathsf{P}(\mathsf{F}(\mathsf{X}) \mathsf{h}) \text{est}(\mathsf{R})) \sqsubseteq \frac{\mathsf{H}}{\mathsf{S}} \text{est}(\mathsf{R})$
every answer the recursion returns is an optimal one

Theorem 9.1

- $\frac{\mathsf{T}^\circ}{\mathsf{S}}$ takes the input apart one step every way; $\mathsf{F}(\mathsf{X})$ solves each part by the recursion X ; h assembles each candidate; $\text{est}(\mathsf{R})$ keeps a best one
- $\sqsubseteq \mathsf{M}$: that never misses the optimum, though all candidates are never listed as H would
- h monotonic on R : a better part never makes the whole worse
- $\sqsubseteq$, not $=$: some optima may not be returned; that the recursion returns anything at all needs T° to stop taking apart — dynamic programming theorem

$$T^\circ F(H)h=H \Rightarrow \frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R))h)\text{est}(R)\sqsubseteq H \quad (9.2)$$

taking the input apart every way T allows ($\frac{T^\circ}{\exists}$), solving each part by M and keeping an optimum ($P(F(M)h) \text{est}(R)$) returns only what H returns



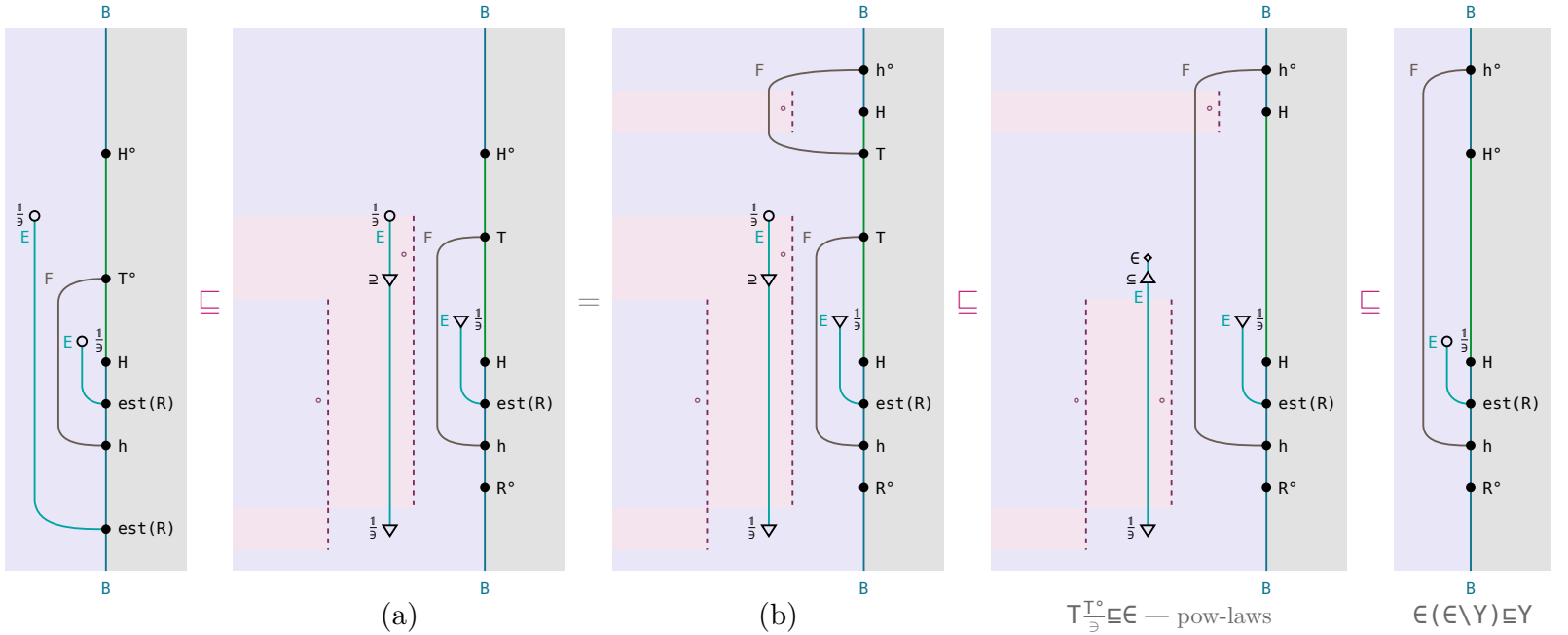
- (a) $P(X)\text{est}(R)\sqsubseteq X$ — est-710
 (b) $\frac{T^\circ}{\exists} \exists = T^\circ$ — pow-laws
 (c) $M \sqsubseteq \frac{H}{\exists} \exists = H$ — est-up
 (d) $T^\circ F(H)h=H$ — hylomorphism theorem

$$T^\circ F(H)h=H \Rightarrow H^\circ \frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R))h)\text{est}(R)\sqsubseteq R^\circ \quad (9.3)$$

for every b that H returns from an input, the step $\frac{T^\circ}{\exists} P(F(M)h) \text{est}(R)$ returns from that input only b' with R relating b' to b

$$T^\circ F(H)h=H \Rightarrow H^\circ \frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R))h)\text{est}(R)\sqsubseteq h^\circ F(H^\circ)F(\frac{H}{\exists} \text{est}(R))hR^\circ$$

from an answer of H , the step's answer is reached by taking H 's parts back, rebuilding them by $F(M)h$, then one R°



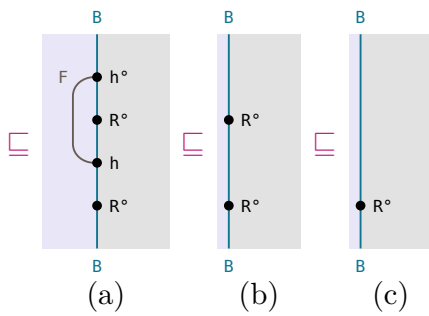
- (a) $P(X)\text{est}(R)\sqsubseteq E \setminus (XR^\circ)$ — est-710
 (b) $H^\circ = h^\circ F(H^\circ)T$ — hylomorphism theorem

$\frac{T^\circ}{\exists} \sqsubseteq E$ — pow-laws

$E(E \setminus Y) \sqsubseteq Y$

$$h^\circ F(H^\circ)F(\frac{H}{\exists} \text{est}(R))hR^\circ \sqsubseteq R^\circ$$

rebuilding an answer of H from its parts, each replaced by M 's, and one R° stays within R°

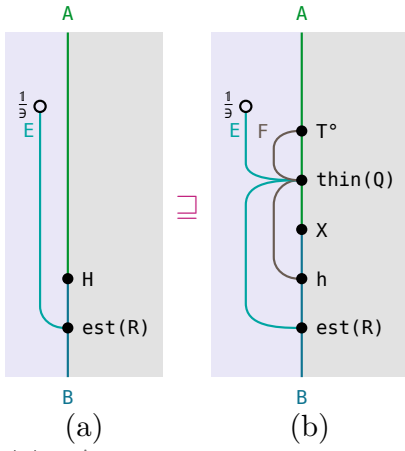


- (a) $H^\circ M \sqsubseteq R^\circ$ — est-up
 (b) h monotonic on R°
 (c) R transitive

$$\text{ThinCondition}(\mathbf{T}, \mathbf{h}, \mathbf{R}, \mathbf{Q}) \Rightarrow (\mu \mathbf{X} : \frac{\mathbf{T}^\circ}{\mathbf{Q}} \text{thin}(\mathbf{Q}) \mathbf{P}(\mathbf{F}(\mathbf{X}) \mathbf{h}) \text{est}(\mathbf{R})) \sqsubseteq_{\frac{\mathbf{H}}{\mathbf{Q}}} \text{est}(\mathbf{R})$$

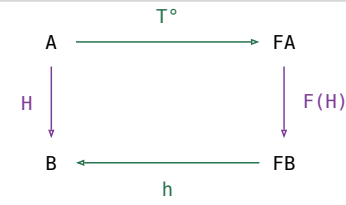
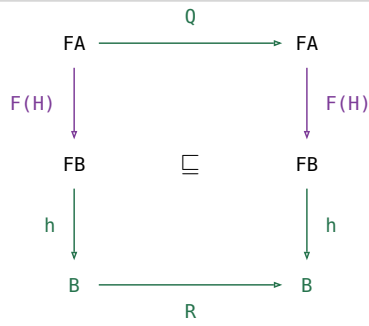
Theorem 9.2

an optimum over everything $\mathbf{H}$ returns is reached by taking the input apart every way $\mathbf{T}$ allows, dropping the parts that can never win, solving each of the rest and keeping one optimum



$$(a) \mathbf{H} \triangleq (\mathbf{T})^\circ (\mathbf{h}) \triangleq \mathbf{H}$$

$$(b) \mathbf{QF}(\mathbf{H}) \mathbf{h} \sqsubseteq \mathbf{F}(\mathbf{H}) \mathbf{hR}, \mathbf{h} \text{ monotonic on } \mathbf{R} \text{ — thin-laws, est-up}$$



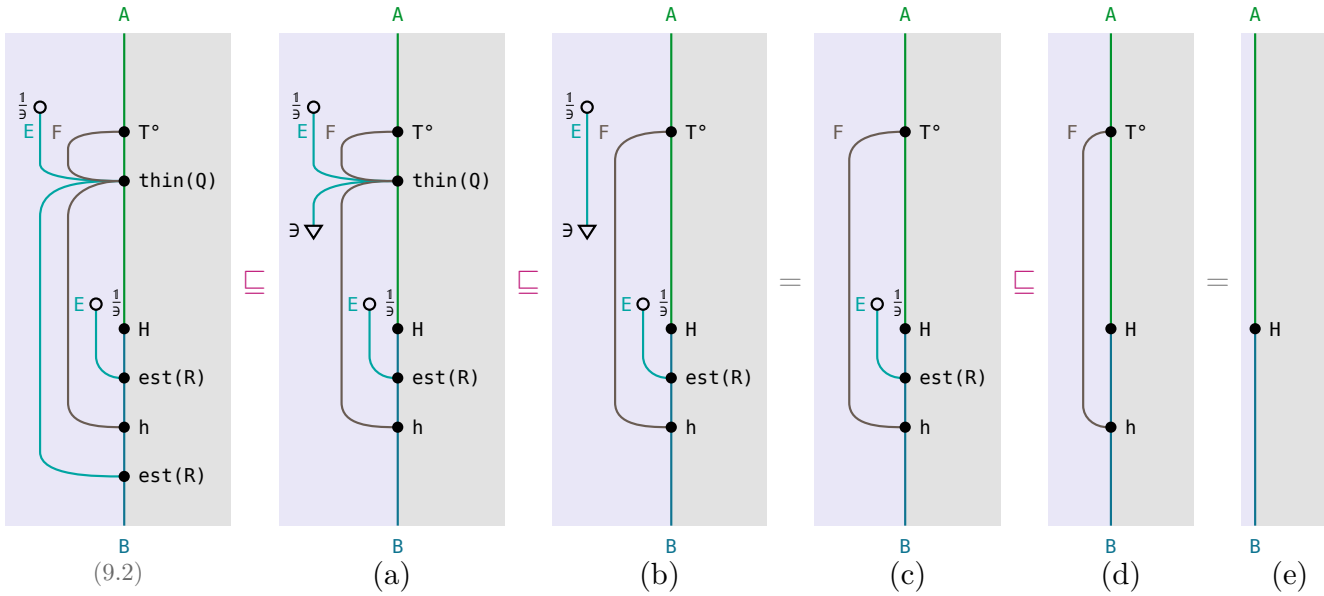
$\mathbf{T}^\circ \mathbf{F}(\mathbf{H}) \mathbf{h} = \mathbf{H}$: taking the input apart once by $\mathbf{T}^\circ$, solving the parts by $\mathbf{F}(\mathbf{H})$ and assembling by $\mathbf{h}$ returns exactly what $\mathbf{H}$ returns

$\mathbf{QF}(\mathbf{H}) \mathbf{h} \sqsubseteq \mathbf{F}(\mathbf{H}) \mathbf{hR}$: every result of thinning by $\mathbf{Q}$, solving the parts by $\mathbf{F}(\mathbf{H})$ and assembling by $\mathbf{h}$ is reached by $\mathbf{R}$ from a result of solving by $\mathbf{F}(\mathbf{H})$ and assembling by $\mathbf{h}$ without $\mathbf{Q}$

$$\mathbf{T}^\circ \mathbf{F}(\mathbf{H}) \mathbf{h} = \mathbf{H} \Rightarrow \frac{\mathbf{T}^\circ}{\mathbf{Q}} \text{thin}(\mathbf{Q}) \mathbf{P}(\mathbf{F}(\frac{\mathbf{H}}{\mathbf{Q}} \text{est}(\mathbf{R})) \mathbf{h}) \text{est}(\mathbf{R}) \sqsubseteq_{\mathbf{H}} \mathbf{H}$$

(9.1.1e)

taking the input apart every way $\mathbf{T}$ allows ($\frac{\mathbf{T}^\circ}{\mathbf{Q}}$), dropping the parts $\text{thin}(\mathbf{Q})$ rejects, solving each rest by $\mathbf{M}$ and keeping an optimum returns only what $\mathbf{H}$ returns



$$(a) \mathbf{P}(\mathbf{X}) \text{est}(\mathbf{R}) \sqsubseteq_{\mathbf{H}} \mathbf{X} \text{ — est-710}$$

$$(b) \text{thin}(\mathbf{Q}) \exists \exists \exists \text{ — thin-laws}$$

$$(c) \frac{\mathbf{T}^\circ}{\mathbf{Q}} \exists = \mathbf{T}^\circ \text{ — pow-laws}$$

$$(d) \mathbf{M} \sqsubseteq_{\frac{\mathbf{H}}{\mathbf{Q}}} \exists = \mathbf{H} \text{ — est-up}$$

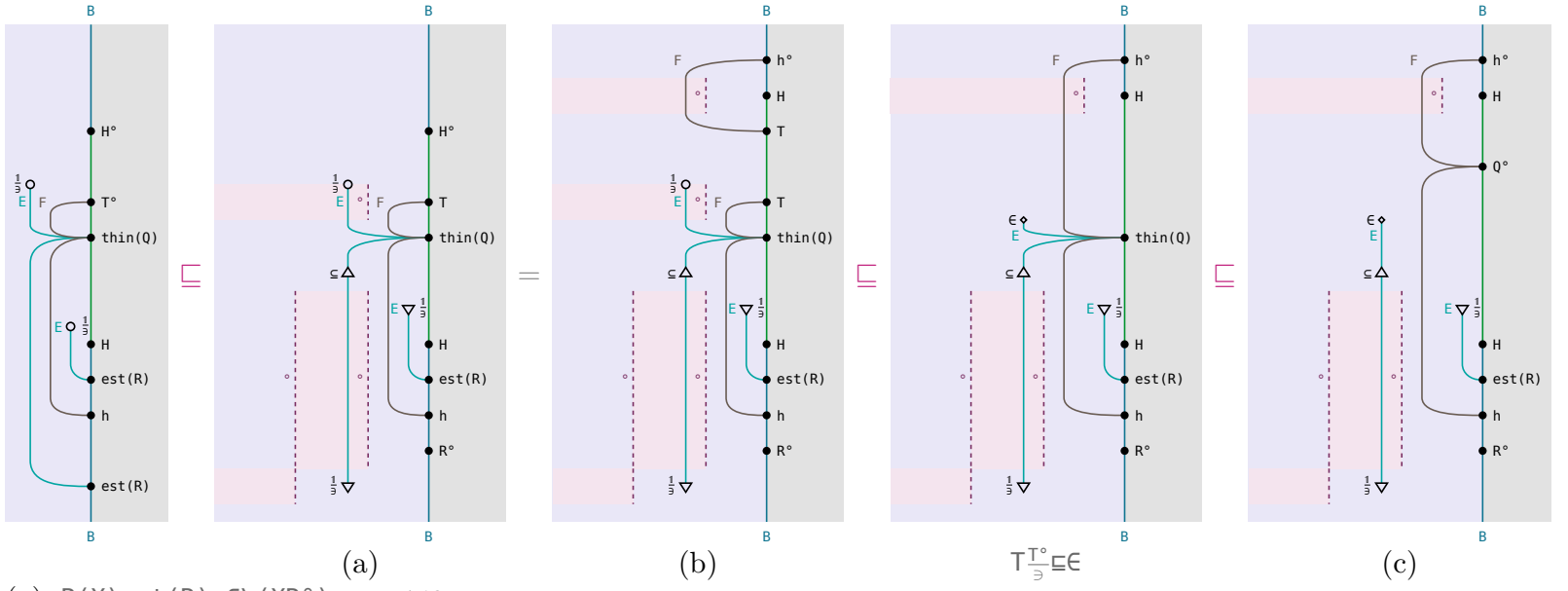
$$(e) \mathbf{T}^\circ \mathbf{F}(\mathbf{H}) \mathbf{h} = \mathbf{H} \text{ — hylomorphism theorem}$$

$$T^\circ F(H)h=H \wedge QF(H)h \sqsubseteq F(H)hR \Rightarrow H^\circ \frac{T^\circ}{\exists} \text{thin}(Q)P(F(\frac{H}{\exists} \text{est}(R))h)\text{est}(R) \sqsubseteq R^\circ \quad (9.1.1f)$$

H° followed by the body at M is $\sqsubseteq R^\circ$: an answer of the body is never worse than an answer of H to the same input

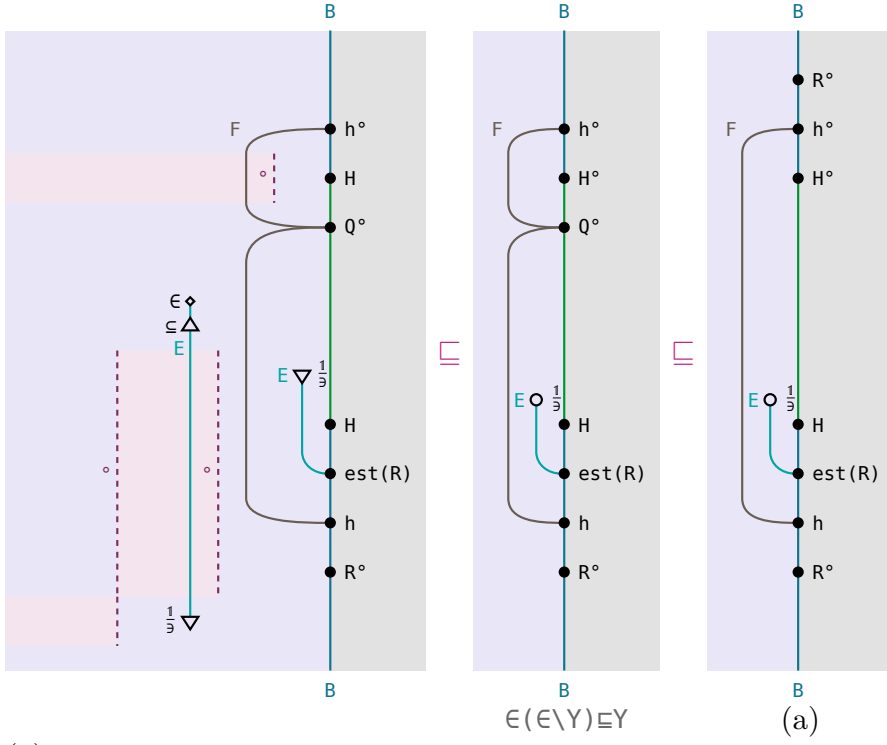
$$T^\circ F(H)h=H \Rightarrow H^\circ \frac{T^\circ}{\exists} \text{thin}(Q)P(F(\frac{H}{\exists} \text{est}(R))h)\text{est}(R) \sqsubseteq h^\circ F(H^\circ)Q^\circ \epsilon (\epsilon \setminus F(\frac{H}{\exists} \text{est}(R))hR^\circ)$$

from an answer of H , the thinned step's answer is reached through H 's parts, a Q° step to a candidate, and that candidate's answer



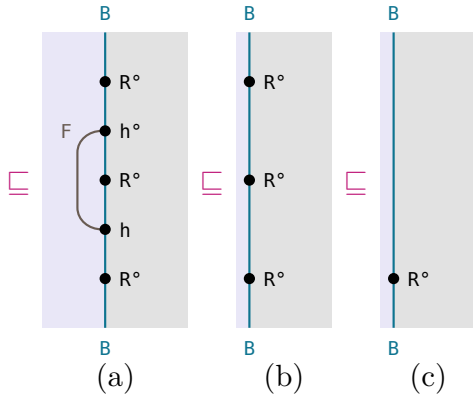
$$QF(H)h \sqsubseteq F(H)hR \implies h^\circ F(H^\circ)Q^\circ \in (\in \setminus F(\frac{H}{3} \text{ est}(R)))hR^\circ \sqsubseteq R^\circ h^\circ F(H^\circ)F(\frac{H}{3} \text{ est}(R))hR^\circ$$

the hypothesis on Q turns the Q° step into an R° step in front of rebuilding by $F(M)h$



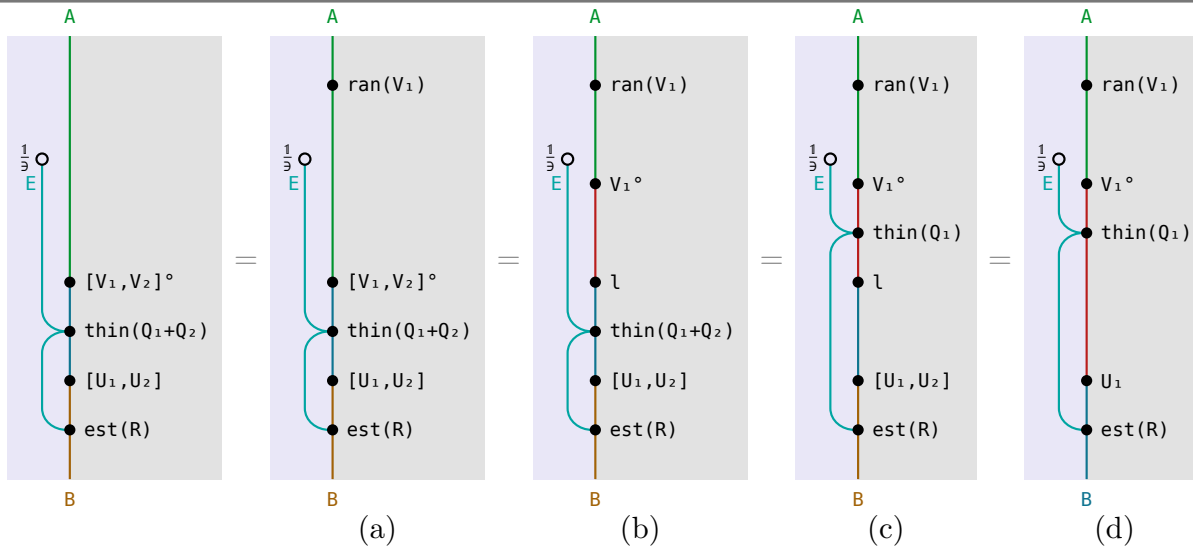
$$R^\circ h^\circ F(H^\circ)F(\frac{H}{3} \text{ est}(R))hR^\circ \sqsubseteq R^\circ$$

rebuilding from parts replaced by M 's, between two R° steps, stays within R°



$$V_2 \ V_1^\circ = \emptyset \implies \frac{[V_1, V_2]^\circ}{\exists} \text{thin}(Q_1 + Q_2) P([U_1, U_2]) \text{est}(R) = \text{ran}(V_1) \frac{V_1^\circ}{\exists} \text{thin}(Q_1) P(U_1) \text{est}(R) \cup \frac{\text{ran}(V_2) \frac{V_2^\circ}{\exists} \text{thin}(Q_2) P(U_2) \text{est}(R)}{\exists} \quad \text{Proposition 9.1}$$

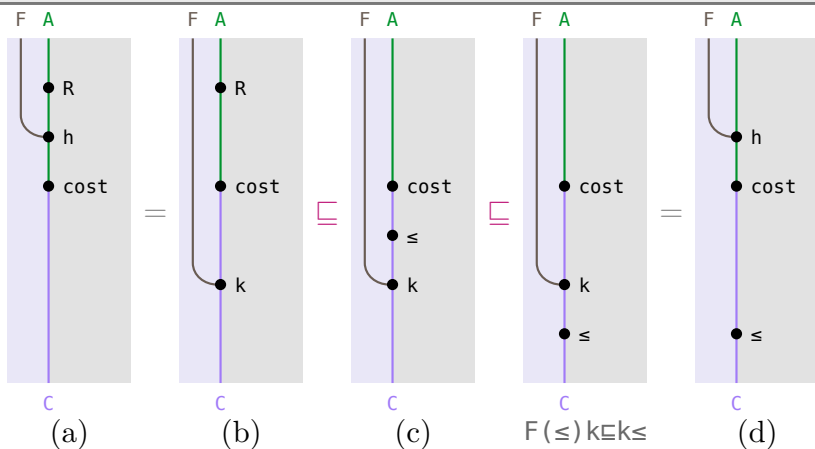
when V_1 and V_2 have disjoint ranges, the body over $[V_1, V_2]$ runs the V_1 problem on inputs V_1 reaches and the V_2 problem on inputs V_2 reaches



- (a) empty off $\text{ran } V_1 \cup \text{ran } V_2$
(b) $\frac{[V_1, V_2]^\circ}{\exists} = \frac{V_1^\circ}{\exists} P(\text{inl})$ on $\text{ran } V_1$
(c) $P(\text{inl}) \text{thin}(Q_1 + Q_2) = \text{thin}(Q_1) P(\text{inl})$
(d) P a relator, $\text{inl}[U_1, U_2] = U_1$

$$\text{Map}(\text{cost}) \wedge R = \text{cost} \leq \text{cost}^\circ \wedge h \text{cost} = F(\text{cost})k \wedge F(\leq)k \leq k \leq \implies \text{MonoAlg}(h, R) \quad \text{Proposition 9.2}$$

if R compares two values by comparing their **costs** under $\leq$, and h then **cost** equals $F(\text{cost})$ then a k monotonic on $\leq$, then h is monotonic on R



- (a) definition of R , shunting
(b) $h \text{cost} = F(\text{cost})k$
(c) $R \text{cost} \leq \text{cost} \leq$
(d) $h \text{cost} = F(\text{cost})k$

(9.1.1h)

the condition	how it is discharged
$F(R)h \subseteq hR$ Proposition 9.2, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $h \text{cost} = F(\text{cost})k$, $F(\leq)k \leq k \leq$	monotonicity when the cost is itself a fold with a step k monotonic on $\leq$
$F(Rn(H^\circ H))h \subseteq hR$ Proposition 9.3, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $h \text{cost} = F(\langle \text{cost}, H^\circ \rangle)k$, $F(\leq \times 1)k \leq k \leq$, H° simple;	monotonicity in context : k may also read the input the part was built from
$QF(H)h \subseteq F(H)hR$ at $Q \triangleq F(U, V)$ Proposition 9.4, U, V preorders, $F(U, R)h \subseteq hR$, $VH \subseteq HR$	both conditions at once, split along the two arguments of a bifunctor

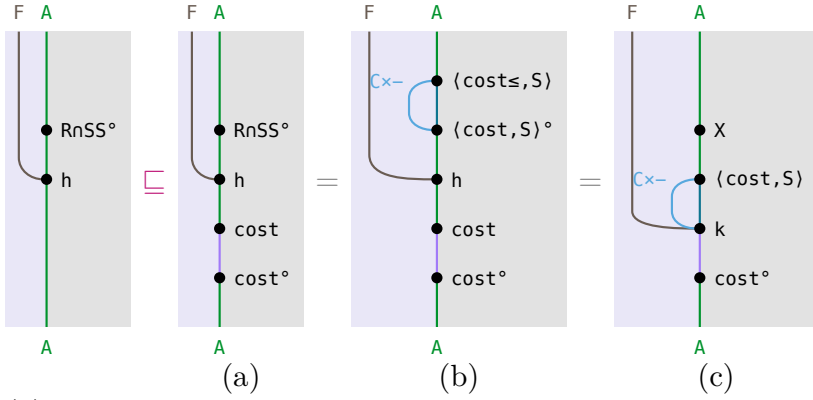
$$\text{Map}(\text{cost}) \wedge \text{Simple}(S) \wedge R = \text{cost} \leq \text{cost}^\circ \wedge \text{hcost} = F(\langle \text{cost}, S \rangle)k \wedge F(\leq \times 1)k \sqsubseteq k \leq \Rightarrow F(RnSS^\circ)h \sqsubseteq hR$$

Proposition 9.3

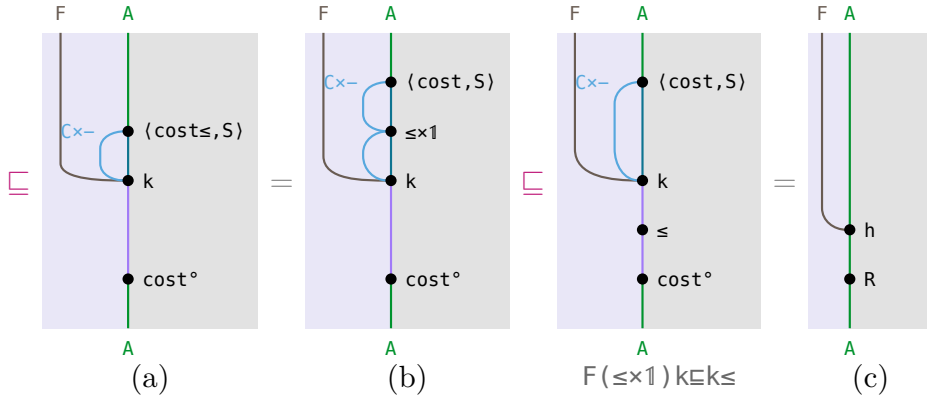
improving each part by R within its S -context, then assembling by h , is below hR

$$\text{Map}(\text{cost}) \wedge R = \text{cost} \leq \text{cost}^\circ \wedge \text{hcost} = F(\langle \text{cost}, S \rangle)k \Rightarrow F(RnSS^\circ)h \sqsubseteq F(\langle \text{cost} \leq, S \rangle \langle \text{cost}, S \rangle^\circ) F(\langle \text{cost}, S \rangle)k \text{cost}^\circ$$

improving the parts within their S -context and assembling by h is below the same improvement on the $(\text{cost}, \text{context})$ bundles, assembled by k



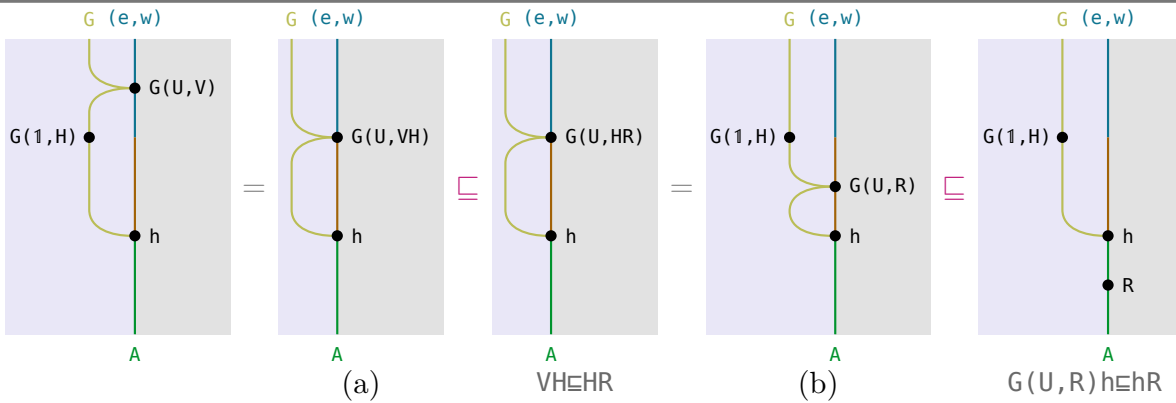
$$\text{Map}(\text{cost}) \wedge \text{Simple}(S) \wedge R = \text{cost} \leq \text{cost}^\circ \wedge \text{hcost} = F(\langle \text{cost}, S \rangle)k \wedge F(\leq \times 1)k \sqsubseteq k \leq \Rightarrow F(\langle \text{cost} \leq, S \rangle \langle \text{cost}, S \rangle^\circ) F(\langle \text{cost}, S \rangle)k \text{cost}^\circ \sqsubseteq hR$$

the improvement on the bundles, assembled by k , is below assembling by h and improving by R 

$$G(U, R)h \sqsubseteq hR \wedge Vh \sqsubseteq hR \Rightarrow G(U, V)G(1, H)h \sqsubseteq G(1, H)hR$$

Proposition 9.4

thinning the parts by U in the first argument and by V in the second $(G(U, V))$, then solving by H and assembling by h , gives only what solving and assembling and then improving by R gives

$$(G(1, H)hR)$$


§9.2. The string edit problem

(9.2a)

- $(\exists a. o = \text{cpy}(a)) \vee (\exists a. o = \text{del}(a)) \vee (\exists a. o = \text{ins}(a))$
an edit operation copies, deletes or inserts one character
- $\text{FX} = 1 + (0p \times X)$
 $F(0p, X)$ is the empty sequence or one operation paired with an X
- $\text{edit} = ([\text{base}, \text{step}])$
 edit is the fold of $[\text{base}, \text{step}]$ over an edit sequence, returning the two strings it reconstitutes
- $\text{length} = ([\text{zero}, \pi_2 \text{ succ}])$
the fold of $[\text{zero}, \pi_2 \text{ succ}]$ counts the operations of a sequence
- $R = \text{length} \leq \text{length}^\circ$
 R relates es to fs when es is no longer than fs
- $V = \text{suffix}^\circ \times \text{suffix}^\circ$
 V relates two pairs of strings when each string of the first is a suffix of the matching string of the second
- $Q(\text{inl}(u), \text{inl}(u))$
 Q is the identity on the **base** summand
- $Q(\text{inr}(p), \text{inr}(q)) \iff V(\pi_2(p), \pi_2(q))$
on the **step** summand Q compares any two operations ($U \triangleq T$) and the two pairs of strings by V
- $\text{empty}(p, q) \iff p = q \wedge p = ([], [])$
 empty is the coreflexive holding only at the pair of two empty strings
- $q \in \text{unstep}(p) \implies \text{step}(\text{inr}(q)) = p$
every decomposition unstep returns is sent back to its input by step

(9.2b)

x	$\alpha(x)$
$\text{inl}(u)$	$\text{wrap}(u)$
$\text{inr}((o, \text{es}))$	$[o] \# \text{es}$

(9.2c)

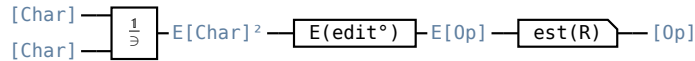
x	$[\text{base}, \text{step}](x)$
$\text{inl}(u)$	$([], [])$
$\text{inr}((\text{cpy}(a), (xs, ys)))$	$([a] \# xs, [a] \# ys)$
$\text{inr}((\text{del}(a), (xs, ys)))$	$([a] \# xs, ys)$
$\text{inr}((\text{ins}(a), (xs, ys)))$	$(xs, [a] \# ys)$

(9.2d)

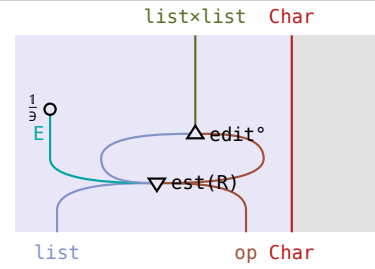
p	$\text{unstep}(p)$
$([], [])$	$[]$
$([a] \# xs, [])$	$[(\text{del}(a), (xs, []))]$
$([], [b] \# ys)$	$[(\text{ins}(b), ([], ys))]$
$([a] \# xs, [b] \# ys)$	if $a = b$ then $[(\text{cpy}, xs, ys)]$ else $[(\text{del}, xs, [b] \# ys), (\text{ins}, [a] \# xs, ys)]$

$$(\mu X : \frac{[base, step]^{\circ}}{3} \text{thin}(Q)P(F(X)[nil, cons])est(R)) \sqsubseteq \frac{edit^{\circ}}{3} est(R), \text{ mle}=(empty \rightarrow nil, unstep \text{list}((1 \times mle)cons) \text{minlist}(R)) \quad (9.2e)$$

a shortest edit sequence from which both strings can be reconstituted is one pass over the two of them, each step copying, deleting or inserting one character and the best sequence for what is left taken from the entries already computed

circuit

the specification — $\triangleq$ edit

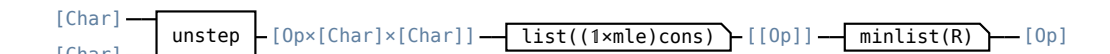
Hinze–Marsden

at $Q \triangleq 1 + (U \times V)$. Monotonicity $F(R) \alpha \sqsubseteq \alpha R$ is Proposition 9.2 at $length \triangleq ([zero, \pi_2 \text{succ}])$ with succ monotonic on $\leq$, so cons is monotonic on R . The thinning condition $\sqsubseteq QF(1, edit^{\circ}) \alpha \sqsubseteq F(1, edit^{\circ}) \alpha R$ over $F(0p, [Char] \times [Char])$ is Proposition 9.4 at $U \triangleq T - F(T, R) \alpha \sqsubseteq \alpha R$, left as an exercise, so any two operations may be compared — and $V \triangleq \text{suffix}^{\circ} \times \text{suffix}^{\circ}$, — $\text{edit}(1 \times \text{suffix}) \sqsubseteq R^{\circ} \text{edit}$, $\text{edit}(\text{suffix} \times 1) \sqsubseteq R^{\circ} \text{edit}$. $\text{edit.suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \Rightarrow BA^* \sqsubseteq C^*B$ cut those to one step each: drop the operation that produced the head, or weaken its cpy to a del , never lengthening the sequence

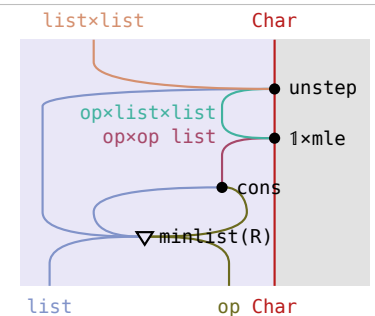
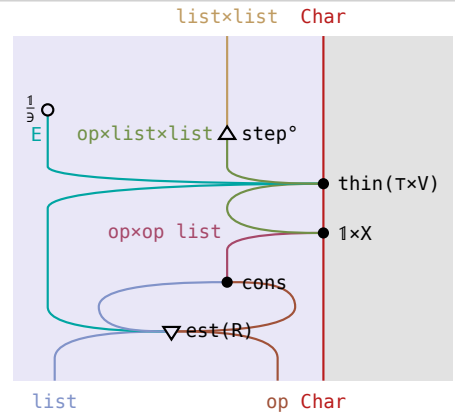
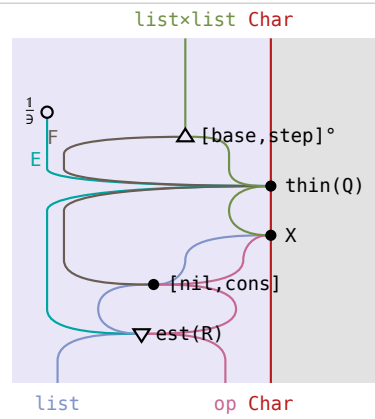
$\sqsubseteq$ Proposition 9.1: base and step have disjoint ranges, so the fixed point splits into one branch per summand — empty , the coreflexive on (xs, ys) with both lists empty, is where base returns



$\sqsubseteq$ Proposition 9.1: base and step have disjoint ranges, so the fixed point splits into one branch per summand — empty , the coreflexive on (xs, ys) with both lists empty, is where base returns

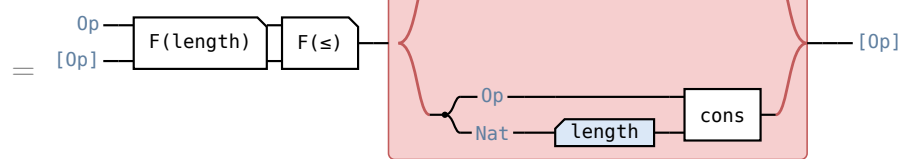
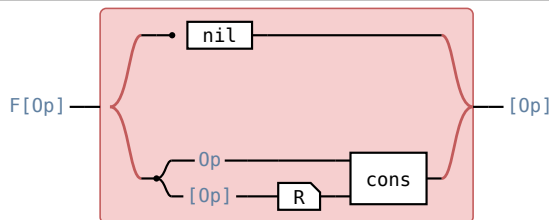


$\sqsubseteq$ unstep implements $\frac{step^{\circ}}{3} \text{thin}(U \times V)$ — at most two decompositions survive, a copy beating a delete or an insert wherever it is available — and $\text{minlist}(R)$ implements $\text{est}(R)$. The same subproblem is solved many times over, so the running time is exponential in the two lengths

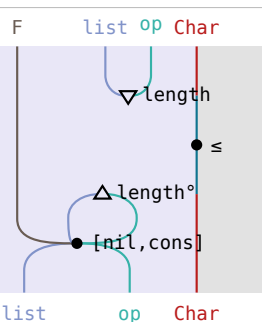
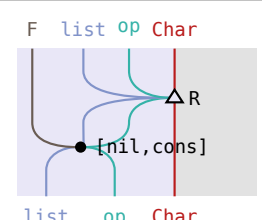


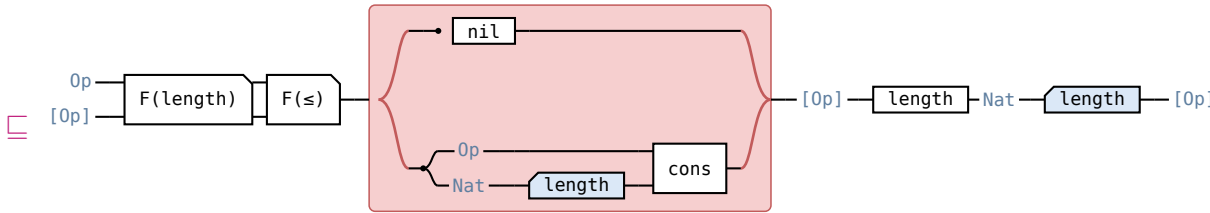
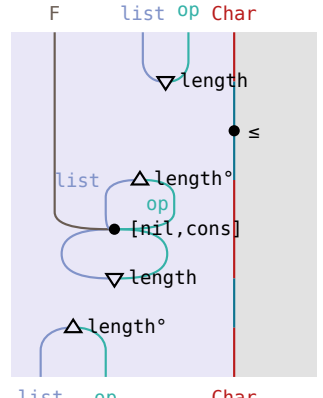
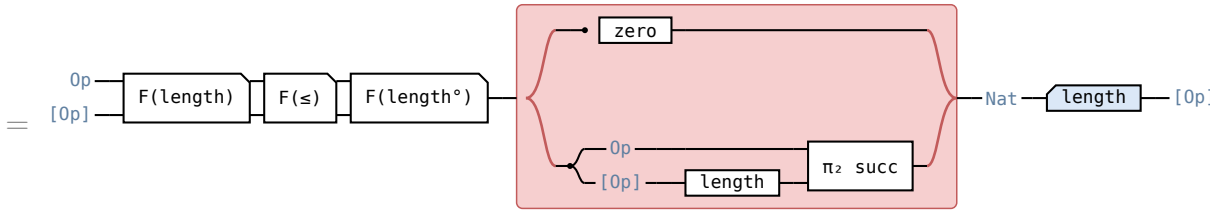
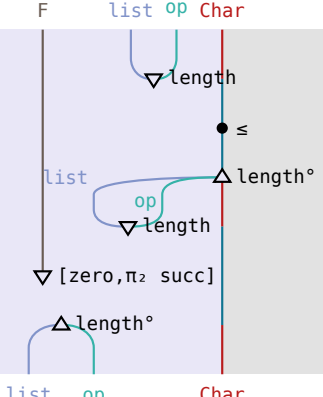
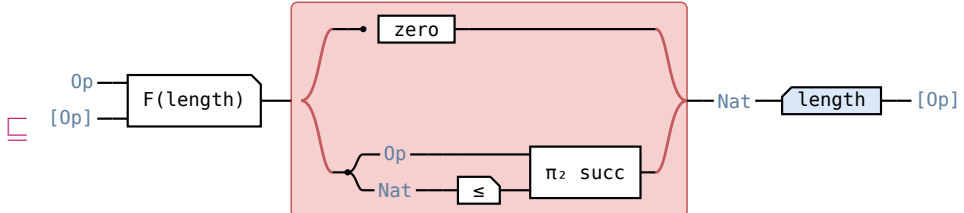
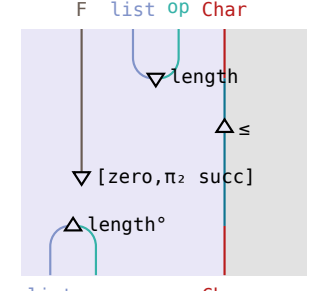
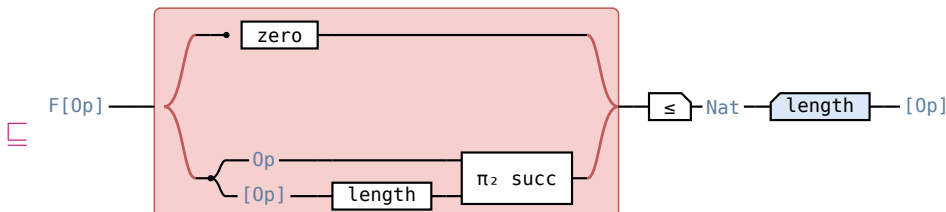
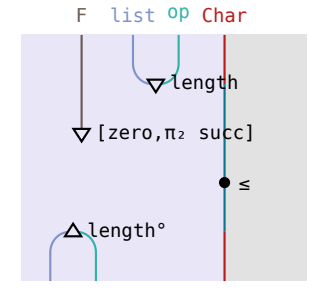
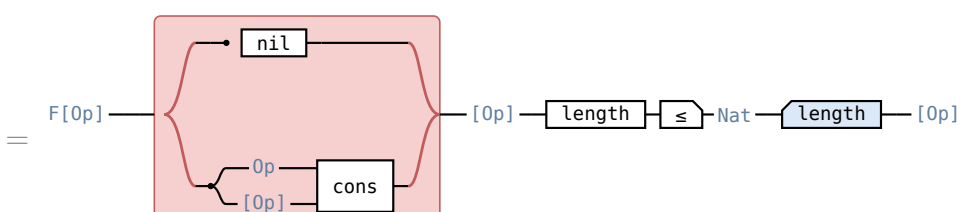
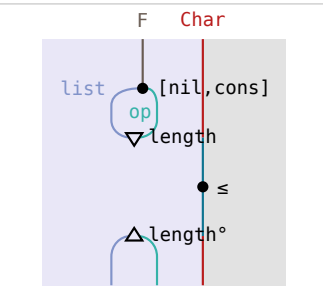
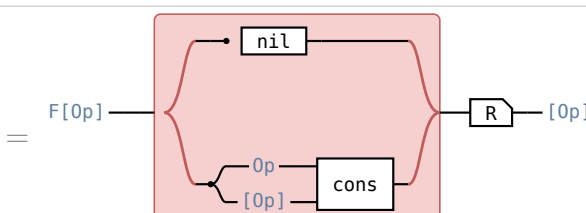
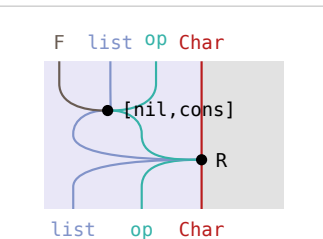
$$F(R)[nil, cons] \sqsubseteq [nil, cons]R \quad (9.2f)$$

comparing the tails by R and then putting an operation in front ($F(R)\alpha$) relates only sequences that R also relates once the operation is in front (αR)

circuit

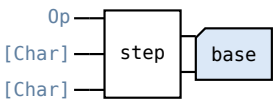
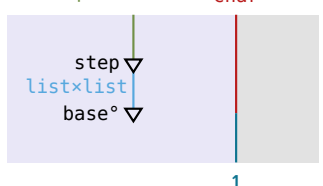
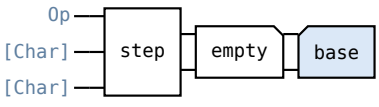
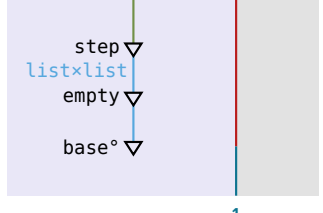
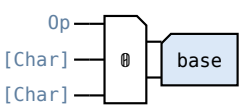
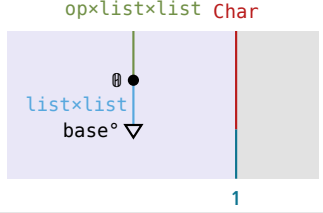
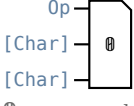
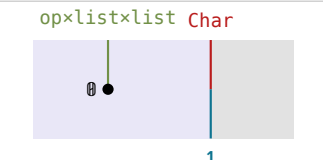
$R \triangleq \text{length} \leq \text{length}^{\circ} \triangleq \text{edit}$; F preserves composition

Hinze–Marsden

circuit	Hinze–Marsden
 length is a map, so entire: $1 \sqsubseteq \text{length } \text{length}^\circ$	
 $\alpha \text{ length} = F(\text{length})[\text{zero}, \pi_2 \text{ succ}] : \text{length}^\Delta([\text{zero}, \pi_2 \text{ succ}]) \triangleq \text{edit}$	
 length is a map, so simple, and F is monotonic: $F(\text{length}^\circ)F(\text{length}) \sqsubseteq F(1) = 1$	
 succ is monotonic on $\leq$: $F(\leq)[\text{zero}, \pi_2 \text{ succ}] \sqsubseteq [\text{zero}, \pi_2 \text{ succ}] \leq$	
 $F(\text{length})[\text{zero}, \pi_2 \text{ succ}] = \alpha \text{ length}$ again	
 $R \triangleq \text{length} \leq \text{length}^\circ \triangleq \text{edit}$	

$QF(edit^\circ)[nil, cons] \sqsubseteq F(edit^\circ)[nil, cons]R$ (9.2g)	
every edit sequence built from a decomposition Q puts above a given one — any operation, each string lengthened at the front — is at least as long as one built from the given one	
circuit	Hinze–Marsden
$Q \triangleq 1 + (U \times V)$ is $F(U, V)$ at $U \triangleq \tau$ — $\triangleq$ edit; and the bifunctor F preserves composition: $F(U, V)F(1, edit^\circ) = F(U, V edit^\circ)$	
Proposition 9.4's second condition $V edit^\circ \sqsubseteq edit^\circ R$ — (9.2h)	
F preserves composition: $F(U, edit^\circ R) = F(1, edit^\circ)F(U, R)$	
Proposition 9.4's first condition $F(\tau, R) \alpha \sqsubseteq \alpha R$, left as an exercise in the book: CONS adds one to both lengths whatever the two operations are	

$\text{edit } V^\circ \sqsubseteq R^\circ \text{ edit}$ (9.2h) cutting a front off either string an edit sequence produces ($\text{edit } V^\circ$) leaves a pair that a sequence no longer produces ($R^\circ \text{ edit}$)	
circuit	Hinze–Marsden
[0p] — edit — V — [Char]	list op Char ∇edit ∇V° list × list Char
[0p] — edit — suffix — [Char] $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, and $\times$ preserves composition	list op Char ∇edit $\nabla \text{suffix} \times 1$ $\nabla 1 \times \text{suffix}$ list × list Char
[0p] — R — edit — suffix — [Char] $\sqsubseteq \text{edit } (\text{suffix} \times 1) \sqsubseteq R^\circ \text{ edit}$: drop the operation that produced the head, or weaken its cpy to an ins. Proved by induction at suffix directly, not through $\text{suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \implies BA^* \sqsubseteq C^*B$ as the book does	list op Char ∇edit $\nabla 1 \times \text{suffix}$ R° list × list Char
[0p] — R — R — edit — [Char] $\sqsubseteq \text{edit } (1 \times \text{suffix}) \sqsubseteq R^\circ \text{ edit}$, the mirror image: cpy weakens to a del	list op Char ∇edit R° R° list × list Char
[0p] — R — edit — [Char] R° is transitive, $\leq$ being so	list op Char ∇edit R° list × list Char

$\text{step } \text{base}^\circ = \emptyset$ (9.2i)	
no pair of strings is both a result of step and the result $([], [])$ of base	
circuit	Hinze–Marsden
	
 base returns only $([], [])$, so $\text{base}^\circ = \text{empty } \text{base}^\circ$	
 $\text{step } \text{empty} = \emptyset$: cpy and del put a character on the left string, ins one on the right	
 $\emptyset$ composed with anything is $\emptyset$	

(9.2j)

$\text{mle}(xs, ys) = \text{head}(\text{column}(xs, ys))$, $\text{column}(xs, ys) = [\text{mle}(u, ys) \mid u \leftarrow \text{tails}(xs)]$ $= \text{column}(xs) = ([\text{fstcol}(xs), \text{nextcol}(xs)])$, $\text{fstcol} = \text{list}(\text{del}) \text{ tails}$ the tabulation: $\text{mle}(xs, ys)$ needs $\text{mle}(u, v)$ for every tail u of xs and v of ys , so the columns are built right to left	
$\text{column}(xs, [b] \# ys) = \text{nextcol}(xs, (b, \text{column}(xs, ys)))$ $= \text{nextcol}(xs)(b, us) = ([\text{base}(b, \text{last}(us)), \text{step}(b)])(xus)$, $xus = \text{zip}(xs, \text{zip}(\text{init}(us), \text{tail}(us)))$ each column is a fold built bottom to top, over xs zipped with the adjacent pairs of the column to its right	
$\text{base}(b, u) = [[\text{ins}(b)] \# u]$ $= \text{step}(b)((a, (u, v)), ws) = (a \rightarrow b \rightarrow [[\text{cpy}(a)] \# v] \# ws, [\text{bmin}(R)([\text{del}(a)] \# w, [\text{ins}(b)] \# u)] \# ws)$ $w = \text{head}(ws)$; these base , step are not edit 's. An entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths	

§9.3. Optimal bracketing

(9.3a)

tree $A ::= \text{tip } A \mid \text{bin } (\text{tree } A, \text{tree } A)$, $FX = A + X^2$, so $F(R) = 1 + R^2$; $h \triangleq [\text{tip}, \text{bin}]$, $\text{flatten} \triangleq ([\text{wrap}, \text{cat}]) : \text{tree } A \rightarrow \text{list}^+ A$, $H = \text{flatten}^\circ$.

$\langle \text{cost}, \text{size} \rangle \triangleq ([\text{opt}, \text{opb}])$, $\text{opt} \triangleq (\text{zero}, \text{st})$, $\text{opb } ((cx, sx), (cy, sy)) = (\text{cb } (sx, sy) + cx + cy, \text{sb } (sx, sy))$.

sb associative, so $\text{size} = \text{flatten } \text{sz}$ for a map sz $R \triangleq \text{cost} \leq \text{cost}^\circ$, $g \triangleq [\text{zero}, (1 \times \text{sz})^2 \text{ opb } \pi_1]$, single the coreflexive on singleton lists.

$\text{splits} \triangleq (\text{inits}^+, \text{tails}^+)$ zip , an implementation of $\frac{\text{cat}^\circ}{\text{zip}}$; $\text{array} \triangleq \text{inits } \text{list}(\text{row})$, $\text{row} \triangleq \text{tails } \text{list}(\text{mct})$, $\text{col} \triangleq \text{inits } \text{list}(\text{mct})$.

$\text{mix} \triangleq \text{zip } \text{list}(\text{bin}) \text{ minlist}(R)$, $\text{next} \triangleq (\pi_1, \text{mix}) \text{ snoc}$, $\text{process} \triangleq ((\text{tip } \text{wrap}) \times 1) \text{ loop}(\text{next})$.

$\text{Assoc}(\text{sb}) \Rightarrow F(\langle \text{cost}, \text{flatten} \rangle)g = [\text{tip}, \text{bin}] \text{cost}$ (9.5)	
pairing each subtree with its cost and its flattening and then applying g gives what building the tree by $[\text{tip}, \text{bin}]$ and taking its cost gives	
circuit	Hinze–Marsden
$F(\text{tree}(A)) \rightarrow \boxed{F(\langle \text{cost}, \text{flatten} \rangle)} \rightarrow F(\text{Int} \times \text{list}^+(A)) \rightarrow \boxed{g} \rightarrow \text{Int}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} A \rightarrow \boxed{\text{zero}} \\ \text{tree}(A) \rightarrow \boxed{\text{cost}} \\ \text{tree}(A) \rightarrow \boxed{\text{flatten}} \rightarrow \boxed{\text{list}^+(A)} \rightarrow \boxed{\text{sz}} \end{array} \right\} \rightarrow \boxed{\text{opb}} \rightarrow \text{Int}$ definition of g; coproducts and products — $\triangleq \text{flatten}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} A \rightarrow \boxed{\text{zero}} \\ \text{tree}(A) \rightarrow \boxed{\text{cost}} \\ \text{tree}(A) \rightarrow \boxed{\text{size}} \\ \text{tree}(A) \rightarrow \boxed{\text{cost}} \\ \text{tree}(A) \rightarrow \boxed{\text{size}} \end{array} \right\} \rightarrow \boxed{\text{opb}} \rightarrow \text{Int}$ $\text{flatten sz} = \text{size}$, since sb is associative — $\triangleq \text{flatten}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} A \rightarrow \boxed{\text{zero}} \\ \text{tree}(A) \rightarrow \boxed{\text{bin}} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{cost}} \rightarrow \text{Int} \\ \text{tree}(A) \rightarrow \boxed{\text{bin}} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{size}} \rightarrow \text{S} \end{array} \right\} \rightarrow \text{Int}$ $\langle \text{cost}, \text{size} \rangle \triangleq ([\text{opt}, \text{opb}])$, at a node $\text{bin}(\text{cost}, \text{size}) = \langle \text{cost}, \text{size} \rangle^2 \text{ opb}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} A \rightarrow \boxed{\text{tip}} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{cost}} \\ \text{tree}(A) \rightarrow \boxed{\text{bin}} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{cost}} \end{array} \right\} \rightarrow \text{Int}$ $\text{tip cost} = \text{zero}$; products, $\langle \text{cost}, \text{size} \rangle \pi_1 = \text{cost}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} A \rightarrow \boxed{\text{tip}} \\ \text{tree}(A) \rightarrow \boxed{\text{bin}} \end{array} \right\} \rightarrow \text{tree}(A) \rightarrow \boxed{\text{cost}} \rightarrow \text{Int}$ coproducts, $[\text{tip cost}, \text{bin cost}] = [\text{tip}, \text{bin}] \text{cost}$	

$F(\leq \times 1) g \sqsubseteq g \leq$ (9.6)	
raising the cost of either subtree ($\leq \times 1$ in both slots) and then applying g returns a value at least what g returns on the costs before raising ($g \leq$)	
circuit	Hinze–Marsden
$F(\text{Int} \times \text{list}^+(A)) \rightarrow \boxed{F(\leq \times 1)} \rightarrow \boxed{g} \rightarrow \text{Int}$	
$A + \text{Int} \times \text{list}^+(A) \times \text{Int} \times \text{list}^+(A) \rightarrow \text{Int}$ $=$ definition of $g \triangleq \text{flatten}$	
$A + \text{Int} \times \text{list}^+(A) \times \text{Int} \times \text{list}^+(A) \rightarrow \text{Int}$ $\sqsubseteq$ definition of opb , and $+$ monotonic	
$A + \text{Int} \times \text{list}^+(A) \times \text{Int} \times \text{list}^+(A) \rightarrow \text{Int}$ $\sqsubseteq$ $\leq$ reflexive, so $\text{zero} \sqsubseteq \text{zero} \leq$; coproducts	
$F(\text{Int} \times \text{list}^+(A)) \rightarrow \boxed{g} \rightarrow \boxed{\leq} \rightarrow \text{Int}$ $=$ definition of $g \triangleq \text{flatten}$	

$$\text{Assoc}(\text{sb}) \Rightarrow (\mu X : \frac{[\text{wrap}, \text{cat}]^\circ}{3} P([\text{tip}, (X \times X) \text{bin}]) \text{est}(R)) \sqsubseteq \frac{\text{flatten}^\circ}{3} \text{est}(R), \text{ mct} = (\text{single} \rightarrow \text{head tip}, \text{ (init col, tail row) mix}) \quad (9.3b)$$

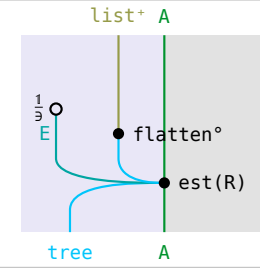
a least-cost bracketing of $a_1 \oplus \dots \oplus a_n$ is read off an array holding one best tree per non-empty segment, each entry built from the column to its left and the row below it

circuit

$$\text{list}^+(A) \rightarrow \frac{1}{3} \rightarrow E(\text{list}^+(A)) \rightarrow E(\text{flatten}^\circ) \rightarrow E(\text{tree}(A)) \rightarrow \text{est}(R) \rightarrow \text{tree}(A)$$

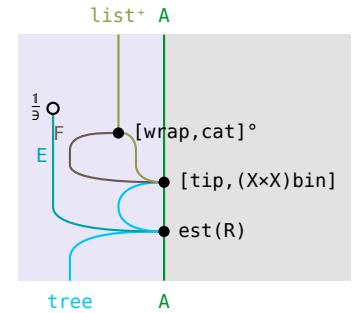
the specification $\triangleq \text{flatten}$

Hinze–Marsden



$$\text{list}^+(A) \rightarrow \frac{1}{3} \rightarrow E(\text{list}^+(A)) \rightarrow E([\text{wrap}, \text{cat}]^\circ) \rightarrow E(\text{list}^+(A)) \rightarrow P([\text{tip}, (X \times X) \text{bin}]) \rightarrow E(\text{tree}(A)) \rightarrow \text{est}(R) \rightarrow \text{tree}(A)$$

split the list in every way, bracket both halves, join. The condition is monotonicity **in context**,
 $\sqsubseteq F(Rn(\text{flatten} \text{ flatten}^\circ)) h \sqsubseteq h R$ — only trees with the same flattening are compared — which is Proposition 9.3 at $H^\circ = \text{flatten}$, a map, with (9.5) $[\text{tip}, \text{bin}] \text{ cost} = (1 + (\text{cost}, \text{flatten})^2) g$ (the cost of a node reads only the cost and the flattening of its two subtrees) and (9.6) $(1 + (\leq \times 1)^2) g \sqsubseteq g \leq (g \text{ monotonic on } \leq \text{ in its two cost arguments})$

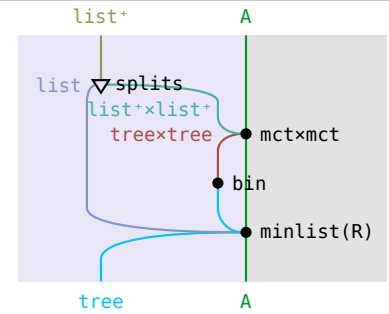


$$\text{list}^+(A) \rightarrow \frac{1}{3} \rightarrow E(\text{list}^+(A)) \rightarrow E(\text{cat}^\circ) \rightarrow E(\text{list}^+(A)^2) \rightarrow P((X \times X) \text{bin}) \rightarrow E(\text{tree}(A)) \rightarrow \text{est}(R) \rightarrow \text{tree}(A)$$

= Proposition 9.1: **wrap** and **cat** have disjoint ranges, and **single** is the coreflexive on singleton lists, where **wrap** returns

$$\text{list}^+(A) \rightarrow \text{splits} \rightarrow [\text{list}^+(A)^2] \rightarrow \text{list}((\text{mct} \times \text{mct}) \text{bin}) \rightarrow [\text{tree}(A)] \rightarrow \text{minlist}(R) \rightarrow \text{tree}(A)$$

$\sqsubseteq \text{splits} \triangleq \langle \text{inits}^+, \text{tails}^+ \rangle$ zip implements $\frac{\text{cat}^\circ}{3}$ and $\text{minlist}(R)$ implements $\text{est}(R)$. Exponential, since the segments of one list overlap



$$\text{mct} = (\text{single} \rightarrow \text{head tip}, (\text{init col}, \text{tail row}) \text{ mix}) \quad (9.7)$$

= the tabulation: **mct** $\times S$ is needed for every non-empty segment $\times S$, so the values are held as an array of rows, $\text{array} \triangleq \text{inits list}(\text{row}), \text{row} \triangleq \text{tails list}(\text{mct}), \text{col} \triangleq \text{inits list}(\text{mct}), \text{mix} \triangleq \text{zip list}(\text{bin}) \text{ minlist}(R)$

$$\text{col} = (\text{single} \rightarrow \text{head tip wrap}, (\text{init col}, \text{tail row}) \text{ next}) \quad (9.8)$$

$$\text{cons col} = (1 \times \text{array}) \text{ process} \quad (9.9)$$

$$\text{row} = (\text{single} \rightarrow \text{head tip wrap}, (\text{mct}, \text{tail row}) \text{ cons}) \quad (9.10)$$

a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop, $\text{next} \triangleq (\pi_1, \text{mix}) \text{ snoc}, \text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next})$

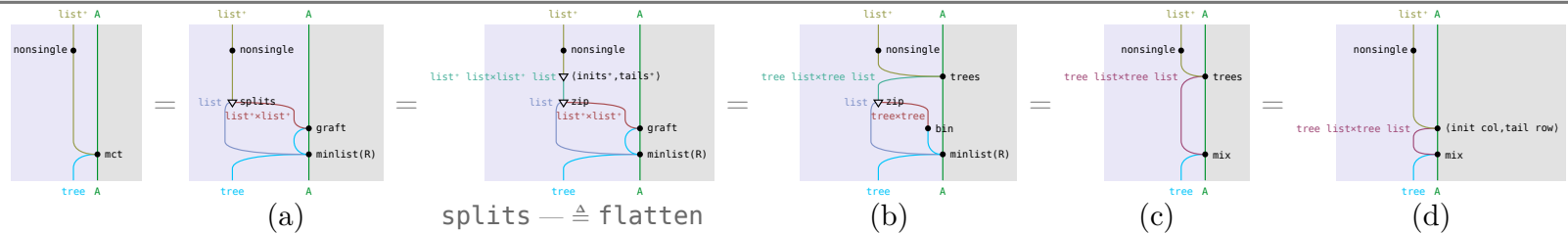
$$\text{array} = ([\text{fstcol}, \text{addcol}]), \text{fstcol} \triangleq \text{tip wrap wrap}$$

$$\text{addcol} \triangleq (\pi_1 \text{ tip wrap}, \text{step}) \text{ cons}, \text{step} \triangleq (\text{process tail}, \pi_2) \text{ zip list}(\text{cons})$$

the program: one fold building the array column by column, cubic in the length of the input

$$\text{nonsingle mct} = \text{nonsingle}(\text{init col}, \text{tail row}) \text{ mix} \quad (9.7)$$

on a list of two or more elements, **mct** gives what **mix** gives on the column of its **init** and the row of its **tail**



- (a) $\text{mct} \triangleq \text{flatten}$
 (b) $\text{graft}, \text{trees}; \text{zip list}(f \times g) = (\text{list}(f) \times \text{list}(g)) \text{ zip}$
 (c) $\text{mix} \triangleq \text{zip list}(\text{bin}) \text{ minlist}(R) \triangleq \text{flatten}$
 (d) $\text{inits}^+ = \text{init inits}, \text{tails}^+ = \text{tail tails}; \text{row}, \text{col} \triangleq \text{flatten}$

nonsingle col=nonsingle(init col,tail row)next (9.8)

on a list of two or more elements, the column is what **next** makes of the column of its **init** and the row of its **tail**

(a) $\text{col} \triangleq \text{flatten}$
 (b) $\text{inits} = (\text{init inits}, 1)$ snoc on non-singletons
 (c) $\text{snoc list}(f) = (\text{list}(f) \times f) \text{ snoc}; \text{col}$
 (d) (9.7) — (9.7); $\text{next} \triangleq (\pi_1, \text{mix}) \text{ snoc} \triangleq \text{flatten}$

nonsingle row=nonsingle(mct,tail row)cons (9.10)

on a list of two or more elements, the row is **mct** of the whole list consed onto the row of its **tail**

(a) $\text{row} \triangleq \text{flatten}$
 (b) $\text{tails} = (1, \text{tail tails})$ cons on non-singletons
 (c) $\text{cons list}(f) = (f \times \text{list}(f)) \text{ cons}; \text{row}$

cons col=(1×array)process (9.9)

the column of **a** consed onto **x** is what **process** makes of **a** and the array of **x**

circuit	Hinze–Marsden
Exercise 9.13 with $k \triangleq \text{cons col}$, $g \triangleq \text{tip wrap}$, $h \triangleq \text{row}$, $f \triangleq \text{next}$: its two equations are (9.8) — (9.8) — at $a: [b]$ and at $a: (u++[b])$, with $[a] \text{ col} = [\text{tip}(a)]$	
products	
definition of process	

tops=(1×array)process tail (9.3c)	
the trees at the top of the new rows are what process makes of a and the array of x , less its first	
circuit	Hinze–Marsden
$\text{list}^+(A) \xrightarrow{A} \boxed{\text{tops}} \rightarrow [\text{tree}(A)]$	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{cons}} \rightarrow \text{list}^+(A) \xrightarrow{\text{init}} \boxed{\text{tail}} \rightarrow [\text{list}^+(A)] \xrightarrow{\text{list(mct)}} \boxed{\text{list(mct)}} \rightarrow [\text{tree}(A)]$ definition of $\text{tops} \triangleq \text{tic list(mct)}$, our name	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{cons}} \rightarrow \text{list}^+(A) \xrightarrow{\text{col}} \boxed{\text{tail}} \rightarrow [\text{tree}(A)]$ $\text{tail list}(f) = \text{list}(f) \text{ tail}$; definition of col	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{array}} \rightarrow \boxed{\text{process}} \rightarrow \boxed{\text{tail}} \rightarrow [\text{tree}(A)]$ (9.9) — (9.9)	

rests=π₂ array (9.3d)	
the rest of each new row is the array of x	
circuit	Hinze–Marsden
$\text{list}^+(A) \xrightarrow{A} \boxed{\text{rests}} \rightarrow [[\text{tree}(A)]]$	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{cons}} \rightarrow \text{list}^+(A) \xrightarrow{\text{init}} \boxed{\text{tail}} \rightarrow [\text{list}^+(A)] \xrightarrow{\text{list(tail row)}} \boxed{\text{list(tail row)}} \rightarrow [[\text{tree}(A)]]$ definition of $\text{rests} \triangleq \text{tic list(tail row)}$, our name	
$= \text{list}^+(A) \xrightarrow{A} \text{list}^+(A) \xrightarrow{\text{array}} \boxed{\text{array}} \rightarrow [[\text{tree}(A)]]$ $\text{tic list(tail)} = \pi_2 \text{ init}$; definition of array	

newrows=(1×array)step (9.3e) the rows of the array of a:x below its first are what step makes of a and the array of x	
circuit	Hinze–Marsden
$\text{list}^+(A) \xrightarrow{A} \boxed{\text{newrows}} \rightarrow [[\text{tree}(A)]]$	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{cons}} \rightarrow \text{list}^+(A) \xrightarrow{\text{inits}} \boxed{\text{tail}} \rightarrow [\text{list}^+(A)] \xrightarrow{\text{list}(\text{row})} [[\text{tree}(A)]]$ definition of newrows = tic list (row), our name	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{cons}} \rightarrow \text{list}^+(A) \xrightarrow{\text{inits}} \boxed{\text{tail}} \rightarrow [\text{list}^+(A)] \xrightarrow{\text{list}(\text{mct}, \text{tail row}) \text{cons}} [[\text{tree}(A)]]$ (9.10) — (9.10) — on the non-singleton tic lists	
$= \text{list}^+(A) \xrightarrow{A} \begin{cases} \text{tops} \\ \text{rests} \end{cases} \rightarrow \boxed{\text{zip}} \rightarrow [\text{tree}(A) \times \text{tree}(A)] \xrightarrow{\text{list}(\text{cons})} [[\text{tree}(A)]]$ $\text{list}(f, g) = (\text{list}(f), \text{list}(g))$ zip; products; definitions of tops and rests	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{array}} \xrightarrow{\text{step}} [[\text{tree}(A)]]$ (9.3c) and (9.3d); products; definition of step	

cons array=(1×array)addcol the array of a consed onto x is what addcol makes of a and the array of x (9.3f)	
circuit	Hinze–Marsden
$= \text{list}^+(A) \xrightarrow{\text{cons}} \text{list}^+(A) \xrightarrow{\text{inits}} [\text{list}^+(A)] \xrightarrow{\text{list}(\text{row})} [[\text{tree}(A)]]$ definition of array	
$= \text{cons } \text{inits} = (\pi_1 \text{ wrap, tic}) \text{ cons, abbreviating cons inits tail by tic}$	
$= \text{list}^+(A) \xrightarrow{\text{newrows}} \text{list}^+(A) \xrightarrow{\text{wrap}} \text{list}^+(A) \xrightarrow{\text{row}} [\text{tree}(A)] \xrightarrow{\text{cons}} [[\text{tree}(A)]]$ cons $\text{list}(f) = (f \times \text{list}(f)) \text{ cons}$; definition of newrows	
$= \text{list}^+(A) \xrightarrow{\text{newrows}} \text{list}^+(A) \xrightarrow{\text{tip}} \text{tree}(A) \xrightarrow{\text{wrap}} [\text{tree}(A)] \xrightarrow{\text{cons}} [[\text{tree}(A)]]$ wrap $\text{row} = \text{tip wrap}$	
$= \text{list}^+(A) \xrightarrow{\text{array}} \text{array} \xrightarrow{\text{addcol}} [[\text{tree}(A)]]$ (9.3e); products; definition of addcol	

§9.4. Data compression

(9.4a)

$[A] ::= \text{nil} \mid \text{snoc } ([A], A), \text{ list}^+ A ::= \text{wrap } A \mid \text{snoc } (\text{list}^+ A, A), \text{ String} = [\text{Char}], \text{ Code} ::= \text{sym Char} \mid \text{ptr} (\text{String}, \text{String}^+), F(\text{Code}, \text{String}) = 1 + (\text{String} \times \text{Code}), \alpha \triangleq [\text{nil}, \text{snoc}].$

$\text{decode} \triangleq ([\text{nil}, \text{extend}]) : [\text{Code}] \rightarrow \text{String}, \text{ extend } (xs, \text{sym } a) = xs \# [a], \text{ extend } (xs, \text{ptr } (ys, zs)) = xs \# zs \text{ when } ys \# zs \text{ is a proper prefix of } xs \# zs; H = \text{decode}^\circ.$

$\text{size} \triangleq ([\text{zero}, \text{distr } [1 \times c, 1 \times p] \text{ plus}])$ with c, p the constant costs of a symbol and a pointer; $R \triangleq \text{size} \leq \text{size}^\circ, Q \triangleq F(\tau + \tau, \text{prefix}^\circ) = 1 + (\text{prefix}^\circ \times (\tau + \tau))$, the two τ on symbols and on pointers.

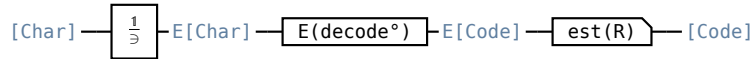
$\text{lrt } ws = \text{est}(\text{prefix}^\circ \times (\tau + \tau)) \{(xs, (ys, zs)) \mid xs \# zs = ws, ys \# zs \text{ a proper prefix of } ws\}$, the longest repeated tail; $\text{reduce } (ws \# [a]) = (zs \# [] \rightarrow [(ws, \text{sym } a), (xs, \text{ptr } (ys, zs))], [(ws, \text{sym } a)])$ with $(xs, (ys, zs)) = \text{lrt } (ws \# [a]).$

$$\theta \leq c \wedge \theta \leq p \Rightarrow (\mu X : \frac{[nil, extend]^\circ}{\exists} \text{thin}(Q)P([nil, (X \times 1)snoc])est(R)) \sqsubseteq \frac{decode^\circ}{\exists} est(R), \quad (9.4b)$$

$$\text{encode} = (\text{null} \rightarrow \text{nil}, \text{reduce list}((\text{encode} \times 1)snoc) \text{minlist}(R))$$

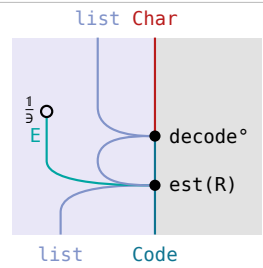
a smallest code sequence decoding to the given string is built from the right, each step emitting the last character as a symbol or ending with a pointer back into what has already been decoded

circuit



the specification $\dashv\equiv$ decode

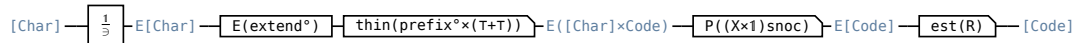
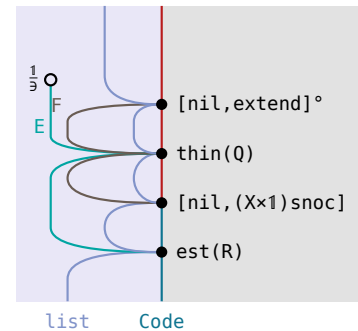
Hinze–Marsden



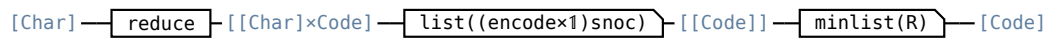
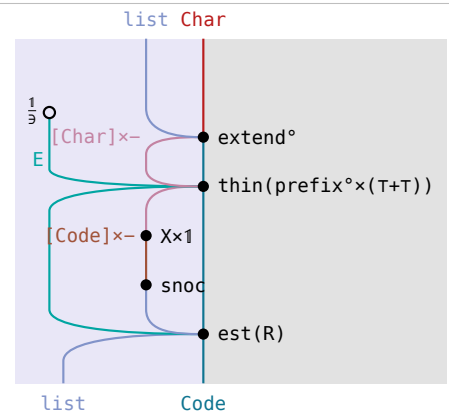
at $Q \triangleq F(T+T, \text{prefix}^\circ) = 1 + (\text{prefix}^\circ \times (T+T))$, the two T on symbols and on pointers.

Monotonicity $F(R) \alpha \sqsubseteq \alpha R$ is routine, the two costs being constants: $(T+T)[c, p] = [c, p]$.

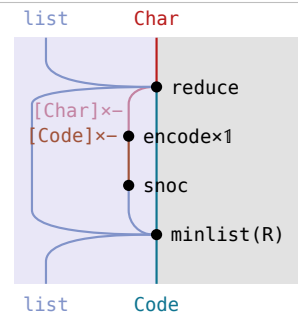
Proposition 9.4 splits the thinning condition in two — $F(T+T, R) \alpha \sqsubseteq \alpha R$, and $\text{decode prefix} \sqsubseteq R^\circ \text{ decode}$, for which $\text{decode init} \sqsubseteq R^\circ \text{ decode}$ is enough: dropping the last character of the output shortens or removes the last code element and never raises the cost. So between a symbol and a pointer nothing can be decided in advance, and between two pointers the longer match wins



Proposition 9.1: nil and extend have disjoint ranges. The decompositions of one string are $\frac{\text{extend}^\circ}{\exists} (ws \# [a]) = \{(ws, \text{sym } a)\} \cup \{(xs, \text{ptr } (ys, zs)) \mid xs \# zs = ws \# [a], ys \# zs \text{ a proper prefix of } ws\}$ — take the last character as a symbol, or end with a pointer



reduce implements $\frac{\text{extend}^\circ}{\exists} \text{thin}(\text{prefix}^\circ \times (T+T))$: thinning leaves at most two, the symbol and the pointer of the longest repeated tail lrt . Again exponential; the book gives no tabulation for it



$\emptyset \leq c \wedge \emptyset \leq p \Rightarrow QF(\text{decode}^\circ)[\text{nil}, \text{snoc}] \sqsubseteq F(\text{decode}^\circ)[\text{nil}, \text{snoc}]R$ (9.4c)	
every code sequence built from a decomposition Q puts above a given one — a code element of the same kind, a longer front string — costs at least as much as one built from the given one	
circuit	Hinze–Marsden
$=$ $Q \triangleq 1 + (\text{prefix}^\circ \times (T+T))$ is $F(T+T, \text{prefix}^\circ) \triangleq \text{decode}$	
$=$ the bifunctor F preserves composition: $F(U, \text{prefix}^\circ)F(1, \text{decode}^\circ) = F(U, \text{prefix}^\circ \text{decode}^\circ)$	
$\sqsubseteq$ Proposition 9.4's second condition $\text{prefix}^\circ \text{decode}^\circ \sqsubseteq \text{decode}^\circ R$: dropping the last sym, or shortens or drops the last pointer. Proved at prefix directly by induction, not through init	
$=$ F preserves composition: $F(U, \text{decode}^\circ R) = F(1, \text{decode}^\circ)F(U, R)$	
$\sqsubseteq$ Proposition 9.4's first condition $F(T+T, R) \alpha \sqsubseteq \alpha R$, left as an exercise in the book: snoc adds the cost of the last element to both sides, and $[c, p](T+T) = [c, p]$	

extend nil^o=⊘ (9.4d) no string is both a result of extend and the result [] of nil	
circuit	Hinze–Marsden
nil returns only [], so $\text{nil}^o = \text{null } \text{nil}^o$	
extend null=⊘: a symbol ends the string with a character, a pointer with its non-empty ZS	
⊘ composed with anything is ⊘	